



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

INGENIERÍA INFORMÁTICA

Documentación

Artemus – Grupo-05

PROFESOR

ALFONSO VILCHEZ DE LAS HERAS

CURSO 2025-2026

Tabla de contenidos

Tabla de contenidos	2
Tabla de ilustraciones	3
1. INTRODUCCIÓN Y CONOCIMIENTO PREVIO	4
2. OBJETIVO GENERAL DE LA FASE	5
3. OBJETIVOS ESPECÍFICOS	6
4. PLANIFICACIÓN Y EQUIPO	7
4.1 Organización del equipo	7
4.2 Metodología de trabajo	7
4.3 Gestión mediante GitHub y ramas	8
5. ANÁLISIS Y DISEÑO	9
5.1 Arquitectura del proyecto	9
5.2 Base de datos	10
5.3 Diseño físico del parque	12
5.4 Esquema Arduino / ESP32	13
6. IMPLEMENTACIÓN	15
6.1 Implementación de la base de datos	15
6.2 Implementación de repositorios	16
6.3 Implementación de servicios	17
6.4 Implementación de la interfaz	18
6.5 Roles, permisos y usuarios	19
6.6 Tickets o solicitudes	20
6.7 Sensorización	21
7. PRUEBAS REALIZADAS	23
8. CONCLUSIÓN	25
9. ANEXO A – REQUISITOS TÉCNICOS Y ENTORNO DE DESARROLLO	26
Requisitos generales del sistema	26
Base de datos y persistencia	27
XAMPP	27
NodeJS	27
Conector MySQL para Python	27
Criptografía	28
Entorno de desarrollo utilizado	28

Tabla de ilustraciones

Ilustración 1: Organización de tareas mediante tablero	7
Ilustración 2: Esquema relacional Artemus Park.	11
Ilustración 3: Boceto físico del parque Artemus Park.	12
Ilustración 4: Esquema de conexión de sensores con Arduino/ESP32.	14

1. INTRODUCCIÓN Y CONOCIMIENTO PREVIO

Artemus Park no parte desde cero, sino que corresponde a la evolución de un sistema previamente desarrollado en fases anteriores del proyecto. Durante las entregas iniciales ya se había diseñado una primera arquitectura funcional de la aplicación, incluyendo una interfaz gráfica desarrollada en Python mediante Flet, una estructura inicial de usuarios y una primera capa de persistencia que posteriormente ha evolucionado hacia una Base de Datos Relacional MariaDB/MySQL.

En esta nueva fase, el proyecto ha evolucionado considerablemente tanto a nivel técnico como estructural. El principal objetivo ha sido consolidar la plataforma hacia un entorno más cercano a una aplicación real, incorporando mejoras relacionadas con la arquitectura del software, la organización interna del proyecto y la integración de tecnologías más robustas.

Entre los avances más importantes realizados durante esta fase destacan la implementación definitiva de la base de datos relacional mediante MariaDB/MySQL, la mejora del sistema de roles y permisos, la actualización de repositorios y servicios, la reorganización del código en capas y el desarrollo de nuevos módulos funcionales.

Además, de la integración de elementos físicos IoT y sensorización, incluyendo el diseño del boceto físico del parque, la distribución de sensores ambientales y la creación de esquemas de conexión para Arduino/ESP32.

También se ha trabajado en la documentación técnica del sistema, diagramas de diseño, pruebas funcionales y evidencias gráficas del desarrollo realizado durante esta etapa.

2. OBJETIVO GENERAL DE LA FASE

El objetivo principal de esta fase consiste en consolidar Artemus Park como una plataforma más completa, organizada y funcional, evolucionando desde un prototipo inicial hacia un sistema más cercano a un entorno real de producción.

Para ello, se ha trabajado en la mejora de la arquitectura general del proyecto, la implementación de una base de datos relacional centralizada y la reorganización de los distintos módulos internos de la aplicación. Del mismo modo, se han añadido nuevas funcionalidades relacionadas con usuarios, roles, permisos, solicitudes y visualización de información.

Otro de los objetivos fundamentales de esta fase ha sido comenzar la integración de hardware y sensorización inteligente mediante el uso de Arduino/ESP32 y distintos sensores ambientales. Esto ha permitido diseñar un modelo inicial de parque inteligente capaz de representar condiciones físicas reales mediante sensores de temperatura, humo, luz, humedad y viento.

Finalmente, esta fase también busca fortalecer la documentación técnica del proyecto, incorporando diagramas, pruebas, capturas y evidencias que permitan reflejar correctamente la evolución del sistema Artemus Park.

3. OBJETIVOS ESPECÍFICOS

Durante esta fase del proyecto se han establecido los siguientes objetivos específicos:

- Actualizar e implementar de forma definitiva la base de datos relacional del sistema utilizando MariaDB/MySQL.
- Mejorar la gestión de usuarios mediante la incorporación de roles, permisos y jerarquías dentro de la aplicación.
- Integrar sensores ambientales y preparar la estructura necesaria para la futura conexión con hardware físico mediante Arduino/ESP32.
- Diseñar el boceto físico del parque Artemus Park representando zonas, caminos y ubicación aproximada de sensores.
- Crear el esquema técnico de conexiones entre sensores y microcontroladores para validar futuras pruebas físicas.
- Actualizar diagramas técnicos relacionados con arquitectura, tablas, relaciones y estructura general del proyecto.
- Mejorar la interfaz gráfica y las funcionalidades principales de la aplicación mediante nuevas vistas y optimización de páginas existentes.
- Documentar pruebas funcionales, evidencias gráficas y resultados obtenidos durante el desarrollo.
- Organizar el proyecto mediante una estructura modular basada en capas para facilitar el mantenimiento y escalabilidad del sistema.

4. PLANIFICACIÓN Y EQUIPO

4.1 Organización del equipo

Durante esta fase, el equipo continuó trabajando con la metodología SCRUM con ciclos de desarrollo de dos semanas, de manera colaborativa siguiendo una distribución de tareas organizada según las necesidades técnicas del proyecto. Cada integrante asumió responsabilidades relacionadas con distintas partes del sistema, incluyendo desarrollo backend, base de datos, interfaz gráfica, documentación y pruebas.

La coordinación del proyecto permitió integrar progresivamente las distintas funcionalidades desarrolladas por los miembros del grupo, manteniendo una estructura organizada y facilitando el avance continuo de la aplicación.

4.2 Metodología de trabajo

Para la gestión del desarrollo se utilizó una metodología de trabajo basada en organización por tareas y seguimiento continuo del progreso. El equipo trabajó mediante herramientas de planificación visual tipo Kanban, permitiendo dividir el proyecto en pequeñas tareas y controlar el estado de cada una de ellas.

Esta metodología permitió mejorar la organización interna, controlar dependencias entre tareas y facilitar la colaboración entre los integrantes del grupo.

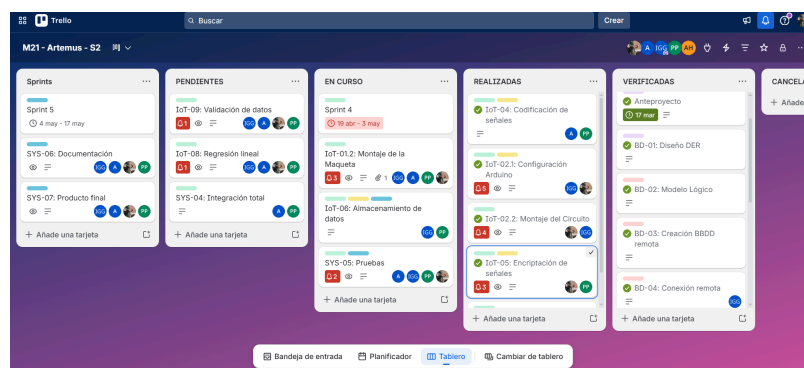


Ilustración 1: Organización de tareas mediante tablero

4.3 Gestión mediante GitHub y ramas

El proyecto se gestionó utilizando GitHub como sistema de control de versiones. Durante esta fase se trabajó principalmente mediante ramas de desarrollo individuales, permitiendo que cada integrante implementa funcionalidades de forma independiente antes de integrarlas en la rama principal del proyecto.

Además, se realizaron múltiples commits y procesos de actualización mediante merge y pull requests, facilitando la integración de cambios realizados sobre módulos como base de datos, repositorios, servicios e interfaz gráfica.

Este sistema permitió mantener una estructura organizada del código y evitar la pérdida de cambios importantes durante el desarrollo colaborativo.

5. ANÁLISIS Y DISEÑO

5.1 Arquitectura del proyecto

La arquitectura actual de Artemus Park ha sido reorganizada durante esta fase con el objetivo de mejorar la escalabilidad, el mantenimiento y la separación de responsabilidades dentro de la aplicación.

El proyecto ha sido desarrollado principalmente en Python utilizando Flet como framework para la interfaz gráfica. La estructura interna del sistema se encuentra dividida en diferentes carpetas y módulos especializados, permitiendo una organización más clara del código y facilitando el trabajo colaborativo entre los integrantes del equipo.

Entre las carpetas principales del proyecto destacan:

- **main:** punto de inicio principal de la aplicación.
- **view:** contiene las distintas páginas y elementos visuales de la interfaz gráfica.
- **service:** incluye la lógica de negocio y procesamiento interno de datos.
- **repository:** encargado de la comunicación entre la aplicación y la base de datos.
- **model:** define las estructuras y modelos de datos utilizados por el sistema.
- **bbdd:** contiene archivos relacionados con la implementación y conexión de la base de datos.
- **config:** almacena configuraciones generales y parámetros del sistema.
- **assets:** incluye imágenes, iconos y recursos gráficos utilizados en la interfaz.

La aplicación sigue una estructura modular basada en capas, separando claramente la interfaz gráfica, la lógica de negocio, el acceso a datos y la persistencia en base de datos. Gracias a esta organización, el sistema resulta más fácil de mantener, actualizar y ampliar en futuras fases del proyecto.

5.2 Base de datos

Durante esta fase se realizó la migración de datos, implementación y consolidación de una base de datos relacional utilizando MariaDB/MySQL como sistema principal de almacenamiento.

En fases anteriores, parte de la información del proyecto era gestionada mediante archivos JSON locales. Sin embargo, en esta fase se ha realizado la migración hacia una Base de Datos Relacional MariaDB/MySQL, centralizando la persistencia de datos y mejorando la seguridad, organización y escalabilidad del sistema. Sin embargo, debido al crecimiento de la aplicación y la necesidad de trabajar con múltiples módulos conectados entre sí, se decidió migrar progresivamente hacia una arquitectura basada en base de datos relacional.

La nueva estructura permite gestionar de forma más segura y organizada la información relacionada con usuarios, sensores, mediciones, solicitudes y jerarquías internas del sistema además de mejorar la escalabilidad de Artemus Park.

Entre las principales tablas implementadas o actualizadas destacan:

- Tabla de usuarios.
- Tabla de roles y permisos.
- Tabla de sensores.
- Tabla de mediciones.
- Tabla de tickets o solicitudes.
- Tabla de jerarquía de usuarios.
- Tablas relacionadas con sensores específicos.

Además, se desarrollaron relaciones entre tablas mediante claves primarias y foráneas, permitiendo mantener la integridad de los datos y evitar inconsistencias dentro del sistema.

Esta nueva arquitectura facilita futuras ampliaciones del proyecto y permite integrar correctamente los datos obtenidos desde sensores físicos y dispositivos IoT.

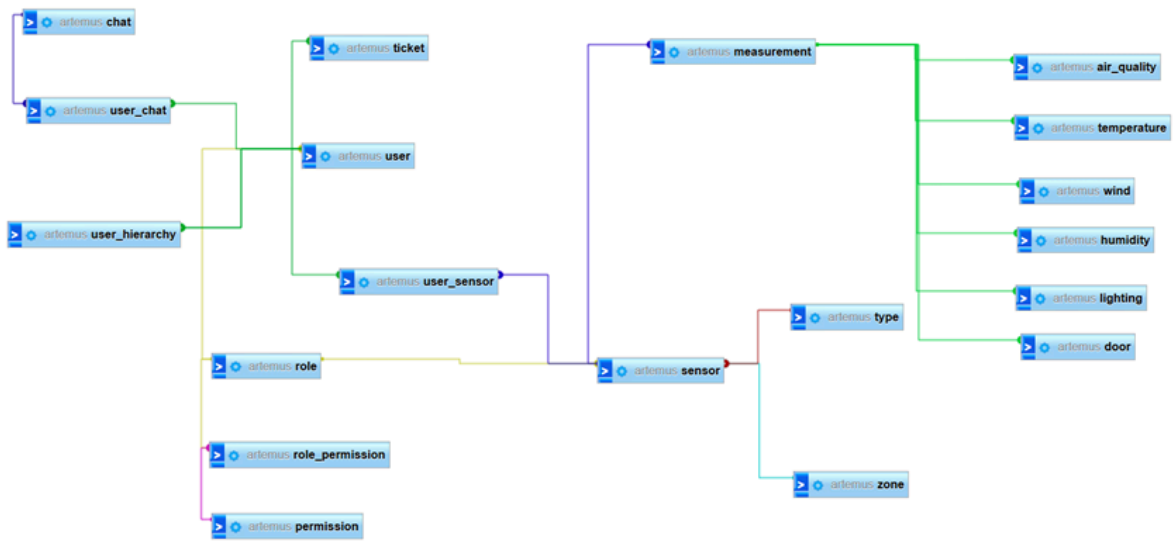


Ilustración 2: Esquema relacional Artemus Park.

5.3 Diseño físico del parque

Como parte del desarrollo del sistema inteligente Artemus Park, se diseñó un boceto físico del parque con el objetivo de representar visualmente la distribución de zonas, caminos y elementos importantes dentro del entorno.

El diseño permite ubicar de forma aproximada distintos sensores ambientales y elementos físicos necesarios para futuras pruebas relacionadas con IoT y sensorización.

Dentro del boceto se representan diferentes tipos de sensores, incluyendo sensores de luz, temperatura, humedad, humo y viento, distribuidos estratégicamente a lo largo del parque. Además, se incluyen caminos, zonas verdes, áreas de agua y puntos de acceso.

Este diseño físico sirve como referencia para futuras implementaciones reales y para planificar correctamente la integración del hardware con el sistema software desarrollado.

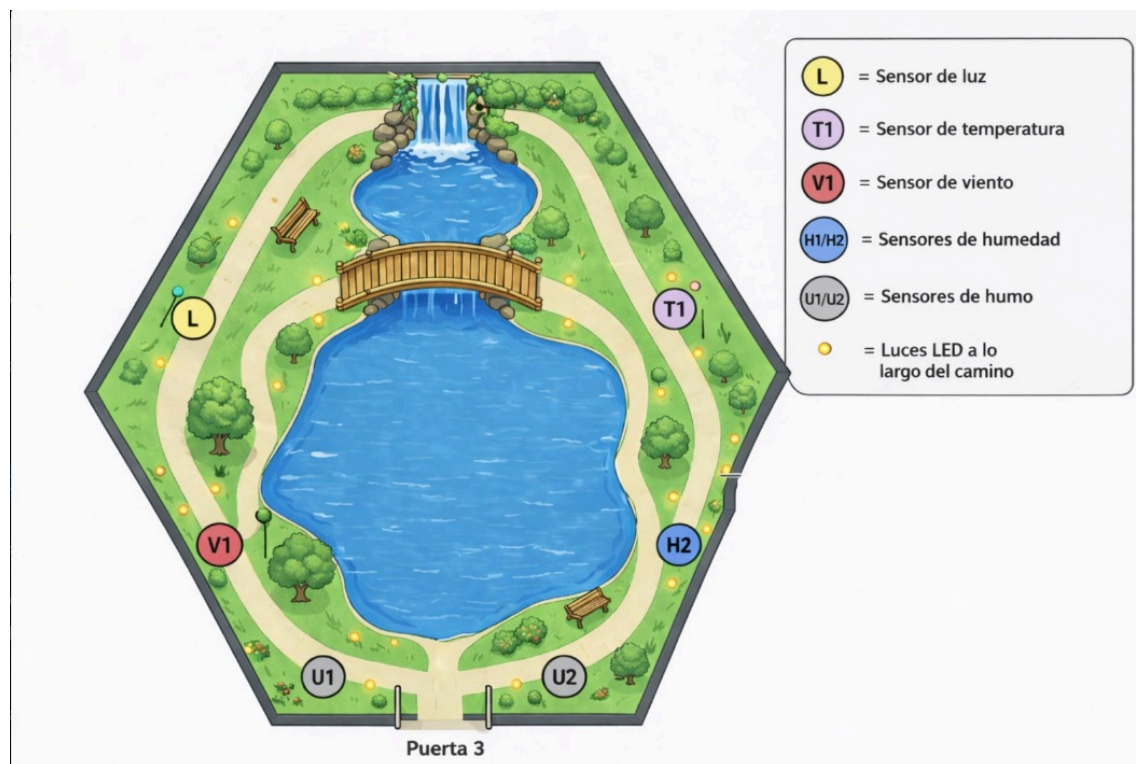


Ilustración 3: Boceto físico del parque Artemus Park.

5.4 Esquema Arduino / ESP32

Durante esta fase también se trabajó en el diseño inicial del esquema de conexiones entre sensores y microcontroladores, utilizando como referencia placas Arduino y ESP32.

El objetivo principal de este esquema es representar cómo se conectarán físicamente los distintos sensores ambientales al sistema electrónico encargado de capturar y enviar los datos hacia la aplicación principal.

En el esquema se incluyen sensores como:

- Sensor DHT22 de temperatura y humedad.
- Sensor MQ de humo/gases.
- Fotorresistencia para detección de luz.
- LEDs de señalización.
- Ventiladores y actuadores.
- Motor DC.
- Resistencias y conexiones de alimentación.

El desarrollo de este esquema permite validar la futura integración física del sistema y facilita la planificación de pruebas reales relacionadas con IoT y automatización.

Además, este modelo servirá como base para implementar futuras lecturas automáticas desde sensores conectados directamente al sistema Artemus Park.

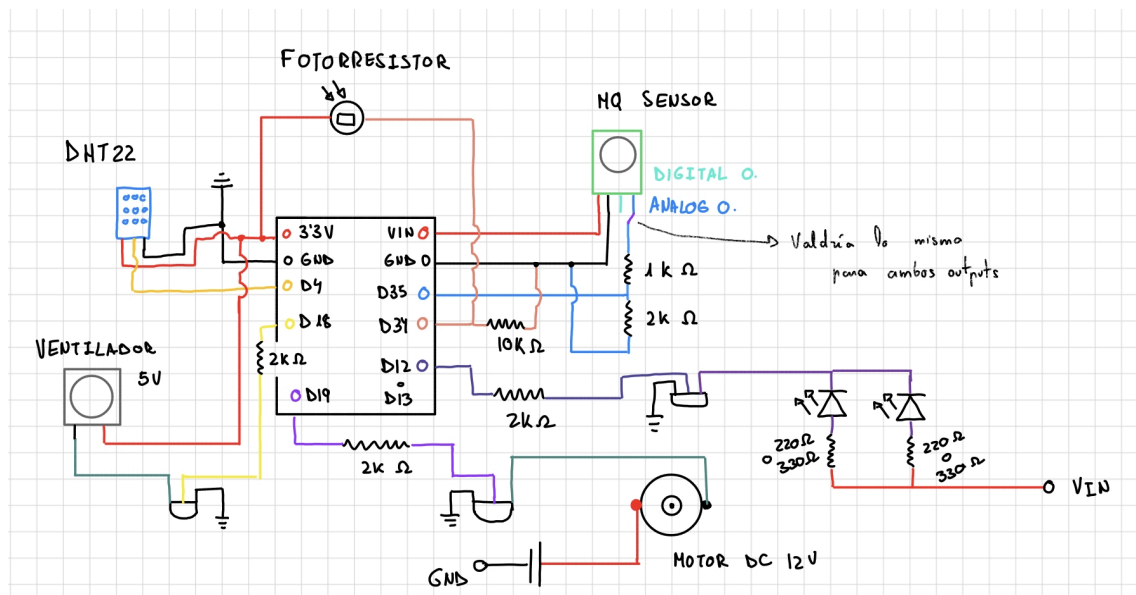


Ilustración 4: Esquema de conexión de sensores con Arduino/ESP32.

6. IMPLEMENTACIÓN

6.1 Implementación de la base de datos

Durante esta fase se realizó una actualización importante sobre la estructura de persistencia del sistema Artemus Park. Para ello, se desarrolló una implementación definitiva basada en MariaDB/MySQL, sustituyendo progresivamente el almacenamiento local mediante archivos JSON utilizado en versiones anteriores del proyecto.

Entre los principales archivos añadidos o modificados destacan:

- `artemus.sql`
- `db_connection.py`
- `conexion_bbdd.py`

El archivo `artemus.sql` contiene la estructura general de la base de datos, incluyendo la creación de tablas, relaciones, claves foráneas, restricciones necesarias y datos de prueba, para garantizar la integridad de la información.

Por otro lado, los archivos `db_connection.py` y `conexion_bbdd.py` permiten establecer la conexión entre la aplicación desarrollada en Python y el sistema gestor de base de datos MariaDB/MySQL.

Gracias a esta implementación, la aplicación puede almacenar y recuperar información de forma más eficiente, segura y escalable.

6.2 Implementación de repositorios

Para mejorar la organización del proyecto, se implementó una capa de repositorios encargada de actuar como intermediaria entre la aplicación y la base de datos.

Estos repositorios centralizan las consultas, inserciones, modificaciones y eliminaciones de información, evitando que la lógica de acceso a datos se encuentre directamente dentro de la interfaz gráfica o de los servicios.

Entre los principales repositorios implementados o actualizados destacan:

- Auth_Repository
- Door_Repository
- Humidity_Repository
- Light_Repository
- Smoke_Repository
- Temperature_Repository
- Wind_Repository
- Requests_Repository

Cada repositorio se especializa en la gestión de un tipo concreto de información, facilitando el mantenimiento y reutilización del código.

Además, esta estructura modular permite futuras ampliaciones del sistema sin afectar directamente al resto de componentes de la aplicación.

6.3 Implementación de servicios

La capa de servicios se desarrolló con el objetivo de gestionar la lógica de negocio de la aplicación y procesar la información antes de enviarla a la interfaz gráfica.

Los servicios permiten separar correctamente el procesamiento interno de datos respecto al acceso a base de datos y la representación visual de la información.

Uno de los servicios principales implementados durante esta fase fue:

- **Dashboard_Service**

Este servicio se encarga de procesar información relacionada con sensores, estadísticas y visualización de datos mostrados dentro del dashboard principal del sistema.

Además, otros servicios internos permiten gestionar autenticación, solicitudes, control de sensores y operaciones relacionadas con usuarios y permisos.

Gracias a esta estructura, la aplicación consigue una arquitectura más organizada y fácil de mantener.

6.4 Implementación de la interfaz

Durante esta fase también se realizaron mejoras importantes sobre la interfaz gráfica de Artemus Park utilizando Flet como framework principal.

Se actualizaron diferentes páginas del sistema tanto a nivel visual como funcional, buscando mejorar la experiencia de usuario y facilitar la navegación dentro de la aplicación.

Entre las páginas principales modificadas o implementadas destacan:

- Login_Page
- Admin_Page
- Dashboard_Page
- History_Page
- Maintenance_Page
- Requests_Page
- Profile_Page
- Chat_Page

Las mejoras realizadas incluyen reorganización visual, optimización de componentes, actualización de navegación y mejora en la visualización de datos provenientes de sensores y base de datos.

Además, se añadieron nuevas funcionalidades relacionadas con usuarios, solicitudes y gestión interna del sistema.

Como la creación de un sistema de mensajería interna como se nos indicó,

6.5 Roles, permisos y usuarios

Uno de los avances más importantes de esta fase fue la mejora del sistema de usuarios, roles y permisos. Se ha implementado un modelo de Control de Acceso Basado en Roles (RBAC), donde los permisos se asocian directamente a los roles. Esto optimiza la gestión de cuentas, ya que al asignar un rol a un nuevo usuario, este hereda automáticamente sus facultades sin necesidad de configuración manual. Además, esta arquitectura escalable permite la inclusión futura de nuevos perfiles sin generar conflictos con el ecosistema existente.

Entre los roles principales implementados destacan:

- Administrador.
- Personal de mantenimiento.
- Usuarios con permisos limitados.
- [Añadir otros roles si existen].

El rol de administrador dispone de acceso completo a funcionalidades relacionadas con gestión de usuarios, solicitudes y visualización global del sistema.

Por otro lado, los usuarios de mantenimiento tienen acceso principalmente a herramientas gestión de sensores, creación de incidencias entre otras.

Esta jerarquía mejora radicalmente la seguridad organizativa. Adicionalmente, la tabla de usuarios ha sido refactorizada para soportar la nueva arquitectura de Mensajería Segura (E2EE). Se han incorporado los campos `public_key` y `private_key` para gestionar la criptografía de los chats, y se han eliminado atributos obsoletos como `connection_status`. Por último, para garantizar la protección total de la identidad, todas las contraseñas se almacenan cifradas mediante algoritmos hash, evitando cualquier exposición directa dentro de la base de datos.

6.6 Tickets o solicitudes

Durante esta fase se mejoró el sistema de tickets orientado a centralizar y registrar las incidencias, mantenimientos y peticiones operativas del parque. Este módulo actúa como el canal oficial de comunicación para la resolución de problemas de infraestructura.

El sistema permite la creación estructurada de solicitudes que incluyen datos clave como la tipología de la incidencia (ej. fallo de sensor, mantenimiento rutinario), la descripción detallada, la fecha de reporte y el usuario emisor (`user_dni`).

Para garantizar un flujo de trabajo eficiente, el módulo se integra estrechamente con el sistema de roles (RBAC):

- Emisión: El personal de mantenimiento y los usuarios pueden registrar nuevas anomalías detectadas en el entorno físico o en el dashboard.
- Resolución: Los administradores disponen de un panel de control dedicado para evaluar, aprobar o rechazar dichas solicitudes, actualizando el estado de la incidencia de forma dinámica (ej. de `PENDING` a `APPROVED` o `REJECTED`).

Toda esta trazabilidad está respaldada por la nueva arquitectura de base de datos relacional (tabla `Ticket`), lo que asegura la persistencia histórica de las operaciones, facilitando futuras auditorías técnicas y mejorando la respuesta operativa ante fallos en la red IoT simulada."

6.7 Sensorización

La sensorización representa uno de los componentes más importantes del proyecto Artemus Park, ya que permite simular un entorno inteligente capaz de interactuar con condiciones ambientales reales.

Durante esta fase se trabajó en la integración conceptual y estructural de distintos sensores ambientales conectados mediante Arduino/ESP32.

Entre los sensores planteados o integrados destacan:

- Sensores de temperatura.
- Sensores de humedad.
- Sensores de luz.
- Sensores de humo.
- Sensores de viento.
- Sensores relacionados con accesos o puertas.

Estos sensores permiten capturar información del entorno y enviarla posteriormente al sistema principal para su procesamiento y visualización.

Además, el diseño físico del parque y el esquema de conexiones permiten preparar futuras pruebas reales relacionadas con automatización e IoT.

6.8 Seguridad y cifrado de comunicaciones

Durante esta fase se han incorporado mejoras importantes relacionadas con la seguridad del sistema de mensajería interna de Artemus Park.

El módulo de chat utiliza cifrado de extremo a extremo, también conocido como End-to-End Encryption, con el objetivo de proteger el contenido de los mensajes enviados entre usuarios. Para ello, se ha implementado un sistema basado en RSA-2048 para la gestión de mensajes privados y claves de usuario.

Además, el sistema incorpora un funcionamiento híbrido, utilizando RSA para las comunicaciones privadas y AES para los mensajes globales. Esta combinación permite mantener un equilibrio entre seguridad y rendimiento.

También se ha trabajado en la protección de las claves privadas de los usuarios mediante su contraseña, evitando que estas queden expuestas directamente dentro del sistema. Esta mejora refuerza la privacidad de las comunicaciones y aporta un mayor nivel de madurez técnica al proyecto.

7. PRUEBAS REALIZADAS

Durante esta fase se realizaron distintas pruebas funcionales con el objetivo de validar el correcto funcionamiento de la aplicación, la integración con la base de datos y las funcionalidades principales del sistema.

Prueba	Objetivo	Resultado esperado	Resultado obtenido	Estado
Conexión con base de datos	Verificar la conexión entre Python y MariaDB/MySQL	Establecer conexión correctamente	La aplicación logró conectarse correctamente a la base de datos	Correcto
Inicio de sesión	Validar autenticación de usuarios	Permitir acceso según credenciales válidas	El sistema validó correctamente usuarios y contraseñas	Correcto
Roles y permisos	Verificar restricciones según rol	Mostrar funcionalidades según permisos	Cada usuario accedió únicamente a sus funcionalidades permitidas	Correcto
Visualización de sensores	Comprobar carga de datos ambientales	Mostrar información de sensores en pantalla	Los datos fueron representados correctamente en el dashboard	Correcto

Gestión de tickets	Validar creación y almacenamiento de solicitudes	Registrar incidencias correctamente	El sistema almacenó tickets en la base de datos	Correcto
Dashboard principal	Verificar carga de estadísticas y datos	Mostrar información general del parque	La interfaz mostró correctamente los datos procesados	Correcto
Navegación entre páginas	Validar funcionamiento de la interfaz	Permitir navegación sin errores	La navegación entre vistas funcionó correctamente	Correcto
Ejecución general	Verificar estabilidad de la aplicación	Ejecutar el sistema sin errores críticos	La aplicación funcionó de forma estable	Correcto

8. CONCLUSIÓN

Durante esta fase, Artemus Park ha evolucionado desde un prototipo inicial hacia un sistema más completo, organizado y cercano a un entorno real. La migración hacia una Base de Datos Relacional MariaDB/MySQL ha permitido mejorar la persistencia de la información, la integridad de los datos y la escalabilidad general del proyecto.

Además, se han reforzado aspectos clave como la gestión de usuarios, roles, permisos, seguridad, la implementación de solicitudes o tickets, la visualización de información mediante el dashboard y la preparación del sistema para integrarse con sensores físicos mediante Arduino/ESP32.

También destaca la mejora del sistema de mensajería interna, incorporando cifrado de extremo a extremo mediante RSA-2048 y AES, lo que aporta una capa adicional de seguridad a las comunicaciones entre usuarios.

En conclusión, esta fase ha permitido consolidar la arquitectura técnica de Artemus Park, mejorar su seguridad, ampliar sus funcionalidades y dejar una base más sólida para futuras integraciones con hardware real y despliegue en un entorno operativo.

9. ANEXO A – REQUISITOS TÉCNICOS Y ENTORNO DE DESARROLLO

Para garantizar el correcto funcionamiento y la estabilidad de Artemus, es indispensable cumplir estrictamente con los siguientes requisitos previos. El equipo de desarrollo no se hará responsable de las incidencias o errores críticos derivados de una configuración que no siga estas especificaciones.

Requisitos generales del sistema

- **Python:** versión 3.10 o superior.
- **RAM mínima:** 4 GB
(8 GB recomendados para un funcionamiento más estable).
- **Sistema Operativo compatible:**
 - Windows 10/11 (64 bits)
 - macOS
 - Distribuciones Linux modernas (Ubuntu 20.04 o superior)
- **Dependencias**

```
pip install -r requirements.txt
```

En el archivo de requirements.txt, se encuentran los siguientes paquetes,

```
flet==0.28.3  
flet-desktop==0.28.3  
cryptography==42.0.5  
Pillow>=11.0.0
```

Base de datos y persistencia

Uno de los principales cambios introducidos en esta fase corresponde a la migración desde un sistema de almacenamiento basado en archivos JSON hacia una arquitectura relacional utilizando MariaDB/MySQL.

Además, se trabajó sobre la integración de un sistema de comunicación interna basado en arquitectura Cliente-Servidor mediante protocolo TCP/IP.

Para el correcto funcionamiento de esta arquitectura se requiere:

XAMPP

- Versión 8.1.10 o superior.

NodeJS

Requerido para el motor de renderizado y el empaquetado de la interfaz de usuario.

- Versión v24.14.1 o superior.

Instalación mediante NVM:

```
nvm install 24.14.1  
nvm use 24.14.1
```

Conector MySQL para Python

Instalación:

```
pip install mysql-connector-python
```

Criptografía

Además de los cambios ya mencionados, el sistema de mensajería incorpora criptografía mediante cifrado de extremo a extremo, también conocido como E2EE. Para los mensajes privados se utiliza RSA-2048, garantizando que únicamente el destinatario legítimo pueda acceder al contenido. Además, el sistema utiliza AES para los mensajes globales, mejorando el rendimiento sin renunciar a la seguridad. Esto garantiza que sólo el destinatario legítimo pueda leer el contenido. Por último hemos implementado también criptografía en las contraseñas para aumentar la seguridad de las cuentas.

Entorno de desarrollo utilizado

Durante el desarrollo del proyecto se utilizaron principalmente las siguientes herramientas:

- PyCharm
- GitHub
- MariaDB / MySQL
- XAMPP
- Flet
- Arduino IDE
- ESP32 / Arduino
- Git Bash y terminal integrada

